

Python Advanced — Book 2

Async · ML · Web Frameworks
Version 2.0 — Deep-Dive Companion Book

Three specialty deep-dives in one book

- **Async & Concurrency** — coroutines, tasks, futures, the event loop internals, cancellation, debugging hung programs, rate-limiting patterns, uvloop, contextvars
- **ML & Data Engineering** — NumPy/Pandas mastery, scikit-learn pipelines, feature engineering, imbalanced classification, production model serving, A/B testing, monitoring
- **Django & FastAPI** — ORM patterns, middleware, DRF, async endpoints, Pydantic v2, dependency injection, auth strategies, deployment, scaling, zero-downtime migrations

Each section: ~20 questions across beginner / intermediate / advanced tiers, with code examples, diagrams, and senior-engineer commentary.

BEGINNER

INTERMEDIATE

ADVANCED

Table of Contents

1.	Async & Concurrency Deep-Dive	B:2 · I:7 · A:10 · 19 total
2.	ML & Data Engineering with Python	B:3 · I:12 · A:10 · 25 total
3.	Django & FastAPI Web Frameworks	B:5 · I:12 · A:10 · 27 total
4.	Cheatsheets	<i>4 quick-reference tables</i>

Total: 71 questions · Beginner: 10 · Intermediate: 31 · Advanced: 30

1. Async & Concurrency Deep-Dive

Goes beyond surface-level threads vs asyncio. Covers internals, common pitfalls, debugging, performance, and production patterns.

Q1. Walk me through exactly what happens when you call ``asyncio.run(main())``.

BEGINNER

ANSWER

1) Creates a new event loop. 2) Sets it as the current loop for the thread. 3) Wraps ``main()`` (a coroutine object — not yet running) in a Task. 4) Runs the loop until the Task completes. 5) Cancels any pending tasks. 6) Closes the loop. `asyncio.run()` should be called ONCE per program — it can't be called from inside an already-running event loop (you'd get `RuntimeError`).

CODE

```
import asyncio

async def main():
    print('hi')

# This:
asyncio.run(main())

# Is roughly:
loop = asyncio.new_event_loop()
asyncio.set_event_loop(loop)
try:
    loop.run_until_complete(main())
finally:
    loop.close()
```

Q2. What's the difference between a coroutine, a task, and a future?

BEGINNER

ANSWER

Coroutine: object returned by calling an ``async def`` function. Inert until awaited or scheduled — it's just a value. Task: a Future subclass that schedules a coroutine on the event loop. Created with `asyncio.create_task()` or `ensure_future()`. Future: a low-level placeholder for an asynchronous result — you set its value, others await it. Most users only deal with coroutines and Tasks.

DIAGRAM

```
async def f(): return 42

c = f()                # coroutine (not running)
t = asyncio.create_task(c)  # task (scheduled)
result = await t        # await it
# 'await c' would also work but task gives you cancel/status
```

Q3. Why does ``await asyncio.sleep(0)`` exist? What's it useful for?

INTERMEDIATE

ANSWER

It yields control back to the event loop without actually waiting. Used for: (1) Giving other tasks a chance to run inside a tight CPU loop. (2) Forcing a coroutine to be a 'real' coroutine for scheduling purposes. (3) Cooperative scheduling break points. WITHOUT it, a long-running coroutine that never awaits will starve the loop.

CODE

```
async def cooperative_compute():
    for chunk in big_data:
        process(chunk)
        await asyncio.sleep(0)    # let others run
```

Q4. How do you run blocking code inside an async function without blocking the loop?

INTERMEDIATE

ANSWER

Use ``asyncio.to_thread()`` (3.9+) or ``loop.run_in_executor(None, fn, *args)``. Both offload to a thread pool. For CPU-bound work, use ``run_in_executor`` with a `ProcessPoolExecutor` instead. NEVER call `requests.get()`, `time.sleep()`, or heavy computation directly in an async function — it freezes every other coroutine.

CODE

```
import asyncio, requests

async def fetch(url):
    # WRONG (blocks the entire event loop)
    # return requests.get(url)

    # RIGHT (offloads to thread)
    return await asyncio.to_thread(requests.get, url)

# CPU-bound:
from concurrent.futures import ProcessPoolExecutor
loop = asyncio.get_event_loop()
result = await loop.run_in_executor(
    ProcessPoolExecutor(), heavy_cpu_function, arg
)
```

Q5. Compare `asyncio.gather`, `asyncio.wait`, and `asyncio.as_completed`.

INTERMEDIATE

ANSWER

`gather`: collects results in input order; fast-fails on first exception (unless `return_exceptions=True`). Use for 'do all of these'. `wait`: returns (done, pending) sets; you inspect each Task. Use when you want a timeout or to stop on `FIRST_COMPLETED`. `as_completed`: yields completed coroutines one at a time, in completion order. Use for streaming results / progress bars. Common bug: `gather` wraps awaitables but doesn't cancel siblings when one fails unless you handle it.

CODE

```
# gather – all-or-nothing
results = await asyncio.gather(f1(), f2(), f3())

# gather with exceptions
results = await asyncio.gather(f1(), f2(), return_exceptions=True)

# wait with timeout
done, pending = await asyncio.wait(
    [task1, task2, task3], timeout=5,
    return_when=asyncio.FIRST_COMPLETED
)
for t in pending: t.cancel()

# as_completed – stream
for coro in asyncio.as_completed([f1(), f2(), f3()]):
    result = await coro
    print('one done:', result)
```

Q6. What is `asyncio.TaskGroup` and why is it preferred over `asyncio.gather` in Python 3.11+?

INTERMEDIATE

ANSWER

`TaskGroup` (PEP 654) provides structured concurrency. If any task raises, ALL siblings are cancelled and exceptions are propagated as an `ExceptionGroup`. `gather()` requires manual cancellation handling and can leak tasks on exception. `TaskGroup` is safer: scope-bound, no orphaned tasks, no silent failures.

CODE

```
# Python 3.11+
async def fetch_all(urls):
    results = []
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(fetch(u)) for u in urls]
    # all tasks done here; any exception → ExceptionGroup
    return [t.result() for t in tasks]

# Exception handling
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(might_fail())
        tg.create_task(also_might_fail())
except* ValueError as eg:
    print('value errors:', eg.exceptions)
```

Q7. How do you cancel a task gracefully?

INTERMEDIATE

ANSWER

Call `task.cancel()` — this schedules a `CancelledError` to be raised inside the task at its next `await`. The task can catch and clean up, then re-raise (don't swallow `CancelledError` silently — it breaks the contract). Use `try/finally` for guaranteed cleanup. `asyncio.shield()` can protect a critical section from cancellation.

CODE

```
async def worker():
    try:
        await long_operation()
    except asyncio.CancelledError:
        print('cleaning up...')
        await cleanup()
        raise # re-raise! never swallow

# To cancel:
task = asyncio.create_task(worker())
task.cancel()
try:
    await task
except asyncio.CancelledError:
    pass
```

Q8. What are async context managers and async iterators?

INTERMEDIATE

ANSWER

Async context manager: implements `__aenter__` and `__aexit__` (both coroutines). Used with ``async with``. Async iterator: implements `__aiter__` and `__anext__` (coroutine). Used with ``async for``. Critical for managing async resources (DB connections, network sessions) and streaming async data.

CODE

```
# Async context manager
class AsyncDB:
    async def __aenter__(self):
        self.conn = await connect()
        return self.conn
    async def __aexit__(self, *exc):
        await self.conn.close()

async with AsyncDB() as db:
    rows = await db.query('...')

# Async iterator
class AsyncCounter:
    def __init__(self, n): self.n = n; self.i = 0
    def __aiter__(self): return self
    async def __anext__(self):
        if self.i >= self.n: raise StopAsyncIteration
        await asyncio.sleep(0.1)
        self.i += 1
        return self.i

async for v in AsyncCounter(5): print(v)
```

INTERMEDIATE

Single-threaded event loop runs coroutines cooperatively. Each `await`` is a suspension point — the coroutine yields control to the loop, which can resume other coroutines waiting on I/O. Under the hood: selectors (epoll/kqueue/IOCP) tell the loop which file descriptors are ready. The loop polls these and resumes appropriate coroutines. No preemption — coroutines must cooperate by awaiting.

[illegible]

ADVANCED

```
# Asyncio
q = asyncio.Queue(maxsize=10)
await q.put(item)
item = await q.get()

# Threads
import queue
q = queue.Queue()
q.put(item) # blocks if full
item = q.get(timeout=5) # blocks if empty

# Processes
from multiprocessing import Queue
q = Queue()
# crosses process boundary via pickle
```

Q11. What's the difference between `asyncio.Lock`, `threading.Lock`, and `multiprocessing.Lock`?

ADVANCED

ANSWER

`asyncio.Lock`: coroutine-safe within one event loop. `acquire()` is awaitable; doesn't block thread, just suspends coroutine. `threading.Lock`: OS mutex; blocks the calling thread. `multiprocessing.Lock`: cross-process mutex (futex on Linux). Use the one matching your concurrency model. Mixing types causes deadlocks or no synchronization at all.

CODE

```
import asyncio
lock = asyncio.Lock()
async def critical():
    async with lock:          # await acquires
        await modify_state()

import threading
tlock = threading.Lock()
def critical_thread():
    with tlock:               # blocks thread
        modify_state()
```

Q12. How do you debug a hung asyncio program?

ADVANCED

ANSWER

(1) Enable debug mode: `asyncio.run(main(), debug=True)` or `PYTHONASYNCIODEBUG=1`. Warns on slow callbacks (>100ms), unawaited coroutines. (2) Use `asyncio.all_tasks()` to list current tasks. (3) Each Task has `get_stack()` and `print_stack()` showing where it's suspended. (4) `py-spy` can attach to a live process and show stacks. (5) Common causes: blocking call in async code (use `to_thread`), missing `await`, deadlock between coroutines.

CODE

```
# Inspect running tasks
import asyncio
for t in asyncio.all_tasks():
    print(t.get_name())
    t.print_stack()

# Debug mode catches:
# - 'coroutine X was never awaited'
# - 'slow callback took N seconds'
# - using non-thread-safe primitives across threads
```


Q13. What is a 'fire and forget' task and what's the risk?

ADVANCED

ANSWER

A task you create but never await. Risk: (1) Exception inside the task is silently swallowed — you only see 'Task exception was never retrieved' on shutdown. (2) The task can be garbage-collected if you don't hold a reference — losing your work. Fix: keep a reference (e.g. in a set, removing on done callback) and use `task.add_done_callback()` to handle errors.

CODE

```
# DANGEROUS
asyncio.create_task(might_fail()) # ref lost → GC → silent

# SAFE
bg_tasks = set()
def schedule(coro):
    t = asyncio.create_task(coro)
    bg_tasks.add(t)
    t.add_done_callback(bg_tasks.discard)
    t.add_done_callback(
        lambda t: t.exception() and log_error(t.exception())
    )
```

Q14. How do you implement a rate limiter with asyncio?

ADVANCED

ANSWER

Semaphore for concurrent request limit. For rate-over-time, use a token bucket with sleep, or ``asyncio_throttle`` / ``aiolimiter``. Don't roll your own incorrectly — leaky bucket has subtle timing issues.

CODE

```
import asyncio

# Limit concurrent in-flight requests
sem = asyncio.Semaphore(10)

async def fetch(url):
    async with sem:
        return await http_get(url)

# Rate over time (simple)
class RateLimiter:
    def __init__(self, rate, per):
        self.rate = rate; self.per = per
        self.allowance = rate
        self.last = asyncio.get_event_loop().time()
        self.lock = asyncio.Lock()

    async def acquire(self):
        async with self.lock:
            now = asyncio.get_event_loop().time()
            elapsed = now - self.last
            self.last = now
            self.allowance += elapsed * (self.rate / self.per)
            if self.allowance > self.rate:
                self.allowance = self.rate
            if self.allowance < 1:
                await asyncio.sleep(
                    (1 - self.allowance) * (self.per / self.rate))
            self.allowance = 0
        else:
            self.allowance -= 1
```

Q15. Explain `loop.call_soon`, `call_later`, `call_at`, and `run_in_executor`.**ADVANCED****ANSWER**

`call_soon(cb, *args)`: schedule a synchronous callback at the next iteration. `call_later(delay, cb)`: schedule after delay seconds. `call_at(when, cb)`: schedule at absolute loop time. `run_in_executor(executor, fn, *args)`: returns a Future for `fn()` running in a thread/process pool. All return Handle objects you can `cancel()`. Useful for bridging sync code into the async world.

CODE

```
loop = asyncio.get_event_loop()

handle = loop.call_later(2.0, print, 'hi')
# handle.cancel() # to cancel

# In executor (most common use)
result = await loop.run_in_executor(
    None,          # default thread pool
    blocking_io_function, arg1, arg2
)
```

Q16. What is uvloop and when should you use it?**ADVANCED****ANSWER**

uvloop is a drop-in replacement for asyncio's event loop, written in Cython on top of libuv. ~2-4x faster than the default loop. Used by aiohttp, FastAPI in production. To install: `pip install uvloop`. To use: `asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())` before run. Limitations: Linux/macOS only (no Windows), can't subclass loop, some debug features differ.

CODE

```
import asyncio
import uvloop

# Option 1: install as default
uvloop.install() # only once at startup
asyncio.run(main())

# Option 2: explicit policy
asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())
```

Q17. How do you propagate context (request ID, user, tracing) across async tasks?

ADVANCED

ANSWER

Use contextvars (PEP 567, stdlib since 3.7). Unlike `threading.local`, `ContextVar` is async-aware — each `Task` gets a copy of the context at creation. Set with `var.set(value)`; read with `var.get()`. `asyncio.run()` and `create_task()` automatically copy the current context.

CODE

```
from contextvars import ContextVar

request_id = ContextVar('request_id', default=None)

async def log_action(action):
    print(f'[{request_id.get()}] {action}')

async def handle_request(req):
    request_id.set(req.id)
    await do_work() # log_action() sees the right id

async def do_work():
    await log_action('working')
```

Q18. Common asyncio anti-patterns and pitfalls?

ADVANCED

ANSWER

(1) Forgetting ``await`` — coroutine is created but never run. Python 3.8+ warns. (2) Using `time.sleep()` instead of `asyncio.sleep()`. (3) Calling `requests.get()` (blocking) in async code. (4) Mixing asyncio and threading without care. (5) Catching `CancelledError` and not re-raising — breaks cancellation. (6) Creating tasks in a loop without storing references. (7) Using `asyncio.wait` without setting `return_when`. (8) Recursive coroutines that grow stack with each `await`. (9) Calling ``asyncio.run`` more than once or inside an existing loop. (10) Forgetting to close sessions/connections (always use `async with`).

Q19. How does aiohttp handle thousands of concurrent connections better than threaded code?

ADVANCED

ANSWER

Each connection is a coroutine, ~few KB overhead (vs OS thread ~8MB stack). The event loop multiplexes via selectors — one OS call returns N ready connections. No context-switch overhead between connections. CPU usage scales with WORK, not connection count. 10k concurrent HTTP connections is routine; 10k threads would crush most servers. Tradeoff: a single CPU-bound coroutine blocks everything.

CODE

```
import aiohttp, asyncio

async def main():
    conn = aiohttp.TCPConnector(limit=1000)
    async with aiohttp.ClientSession(connector=conn) as s:
        tasks = [s.get(f'https://api.example.com/{i}')
                  for i in range(10000)]
        responses = await asyncio.gather(*tasks)
```

2. ML & Data Engineering with Python

NumPy, Pandas, scikit-learn fundamentals, data pipelines, performance, and ML system design.

Q1. Why NumPy is dramatically faster than pure Python for numerical work?

BEGINNER

ANSWER

(1) Contiguous memory: ndarrays are typed C arrays — no per-element Python object overhead. A million ints in a list = a million boxed PyObjects (~28MB); same in np.array = ~8MB. (2) Vectorization: operations execute in compiled C with SIMD, no Python interpreter loop. (3) Broadcasting: shape-aware ops without copies. Rule: if you write a `for` loop over a NumPy array, you're probably doing it wrong.

CODE

```
import numpy as np

# Slow: 10x slower
result = [x * 2 for x in py_list]

# Fast: vectorized
result = arr * 2

# Broadcasting (no copy)
matrix + row_vector # adds row to each row
```

Q2. Explain NumPy's axis parameter.

BEGINNER

ANSWER

axis=0 means 'along rows' = collapse rows = result has shape of columns. axis=1 means 'along columns' = collapse columns. axis=None aggregates over all elements. Easiest to remember: axis=N collapses the Nth dimension.

CODE

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])

a.sum() # 21 (everything)
a.sum(axis=0) # [5, 7, 9] — sum across rows
a.sum(axis=1) # [6, 15] — sum across cols
a.mean(axis=0) # column means
```

Q3. Pandas Series vs DataFrame — when do you use each?

BEGINNER

ANSWER

Series: 1D labeled array (a single column with an index). DataFrame: 2D table — many Series sharing the same index. Index is the powerful part: alignment, joins, fast lookups, time-series operations. Most workflows: read CSV/Parquet → DataFrame; operate column-wise (Series); aggregate / groupby; output.

CODE

```
import pandas as pd

s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s['b']    # 2 by label

df = pd.DataFrame({
    'name': ['alice', 'bob'],
    'age': [30, 25]
})
df['age']    # Series
df.iloc[0]  # first row as Series
```

Q4. Difference between .loc, .iloc, and chained indexing (df[a][b]).

INTERMEDIATE

ANSWER

loc: label-based. iloc: integer position-based. df[col] returns a Series; df[col][row] is 'chained' — works for reads but FAILS silently for writes due to view-vs-copy ambiguity (the famous SettingWithCopyWarning). Always use df.loc[row, col] = value for assignment.

CODE

```
df.loc[0, 'age']    # label-based
df.iloc[0, 1]       # position-based

# WRONG (chained indexing)
df['age'][0] = 31    # may or may not work, warns

# RIGHT
df.loc[0, 'age'] = 31
```

Q5. Explain Pandas groupby — the split-apply-combine pattern.

INTERMEDIATE

ANSWER

Three steps: split data into groups by key, apply a function to each group, combine results. `agg()` for reduction (mean, sum). `transform()` returns same-shape result (good for normalization within groups). `apply()` is most general but slowest. `Sort=False` for speed when order doesn't matter.

CODE

```
# Aggregate
df.groupby('city')['salary'].mean()

# Multiple aggregations
df.groupby('city').agg({
    'salary': ['mean', 'std'],
    'age': 'max'
})

# Transform – z-score within group
df['z'] = df.groupby('city')['salary'].transform(
    lambda x: (x - x.mean()) / x.std()
)

# Apply (most flexible, slowest)
df.groupby('city').apply(custom_function)
```

Q6. How do you handle missing data in Pandas?

INTERMEDIATE

ANSWER

Detect: `isna()` / `notna()`. Drop: `dropna(subset=cols, how='any|all', thresh=N)`. Fill: `fillna(value)`, `fillna(method='ffill'/'bfill')`, or `fillna(df.mean())`. For ML, often impute with median/mode before training. Modern Pandas distinguishes `np.nan` (float NaN) from `pd.NA` (typed nullable) — `pd.NA` propagates more consistently.

CODE

```
df.isna().sum()          # missing count per column
df.dropna(thresh=3)      # keep rows with >=3 non-NA
df.fillna({'age': df['age'].median(),
           'name': 'unknown'})
df['x'].ffill()          # forward fill
```

Q7. Common Pandas performance pitfalls and how to avoid them.

INTERMEDIATE

ANSWER

(1) Avoid `iterrows()` and `itertuples()` — they're 100-1000x slower than vectorization. (2) Use `.at[i, col]` for single-cell access, not `.loc`. (3) Avoid chained operations that copy: `df = df.dropna()`; `df['x'] = ...` — use `inplace=True` or assign once. (4) `df.apply(fn)` is slow; prefer vectorized ops or `np.vectorize` / `numba`. (5) Categoricals save memory and speed up groupby. (6) Use `pd.read_csv(dtype={...})` to avoid object dtype which is slow.

CODE

```
# Slow
for i, row in df.iterrows():
    df.loc[i, 'total'] = row['a'] + row['b']

# Fast (vectorized)
df['total'] = df['a'] + df['b']

# Categoricals – big memory win on repeated strings
df['city'] = df['city'].astype('category')
```

Q8. When to use NumPy vs Pandas?

INTERMEDIATE

ANSWER

NumPy: pure numerical computation, fixed-shape arrays, when you don't need labels/index. Faster and lower memory for raw math. Pandas: tabular data, mixed types, labeled axes, joins, group-by, time series. Pandas is built ON NumPy — operations on numeric columns are numpy under the hood. Don't put a NumPy array in a Pandas DataFrame just to do math on it.

Q9. What is broadcasting in NumPy? Show the rules.

INTERMEDIATE

ANSWER

Broadcasting lets ops work on different-shaped arrays by virtually stretching them. Rules: (1) Align shapes from the right. (2) Each dimension must match OR be 1 OR not exist. (3) Dimensions of size 1 are 'stretched' to match. No actual copy is made.

CODE

```
import numpy as np
a = np.arange(12).reshape(3, 4) # (3,4)
b = np.array([10, 20, 30, 40]) # (4,)

a + b # b broadcasts to (3,4)
# rows are [0+10, 1+20, 2+30, 3+40], etc.

c = np.array([[100], [200], [300]]) # (3, 1)
a + c # c broadcasts to (3, 4)
# adds 100 to row 0, 200 to row 1, etc.

# Incompatible
d = np.arange(5)
# a + d → ValueError: shapes (3,4) (5,) can't broadcast
```

Q10. How do you handle large CSV files that don't fit in memory?

INTERMEDIATE

ANSWER

(1) Chunked reading: `pd.read_csv(path, chunksize=N)` returns an iterator of DataFrames. Process and aggregate chunk by chunk. (2) dtype optimization: specify dtypes upfront to reduce memory. (3) Convert to Parquet — columnar, compressed, supports predicate pushdown. (4) Use Dask, Polars, or DuckDB for out-of-core processing. (5) For pure scanning, raw csv module is even more memory-efficient than Pandas.

CODE

```
# Chunked aggregation
total = 0
for chunk in pd.read_csv('huge.csv', chunksize=100_000):
    total += chunk['amount'].sum()

# Polars – much faster for huge data
import polars as pl
df = pl.scan_csv('huge.csv').filter(
    pl.col('amount') > 0
).group_by('city').agg(pl.col('amount').sum()).collect()
```

Q11. What's the typical scikit-learn workflow?

INTERMEDIATE

ANSWER

1) Load data (Pandas/NumPy). 2) Train/test split with `train_test_split`. 3) Preprocess: scaling (`StandardScaler`), encoding (`OneHotEncoder`), impute (`SimpleImputer`) — fit on TRAIN, transform on both. 4) Build pipeline: `Pipeline([('prep', ...), ('model', ...)])`. 5) Train: `pipeline.fit(X_train, y_train)`. 6) Evaluate: `cross_val_score`, `classification_report`. 7) Hyperparameter tune: `GridSearchCV` / `RandomizedSearchCV`. 8) Persist: `joblib.dump(pipeline, 'model.pkl')`.

CODE

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipe = Pipeline([
    ('scale', StandardScaler()),
    ('model', LogisticRegression())
])

scores = cross_val_score(pipe, X_train, y_train, cv=5)
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))
```

Q12. Why MUST you fit scalers/encoders on train data only, not on full data?

INTERMEDIATE

ANSWER

Data leakage. If you fit `StandardScaler` on the full dataset, the test set's mean/std bleeds into the scaler — your model is implicitly seeing test data during training. This inflates test scores and hides real-world performance. Always: split first, fit preprocessor on train, transform both. Pipelines handle this automatically in cross-validation.

Q13. Explain overfitting vs underfitting and how to detect each.

INTERMEDIATE

ANSWER

Overfitting: model memorizes training data but generalizes poorly. Train accuracy >> test accuracy. Symptoms: huge gap between train and validation curves. Fix: regularization, more data, simpler model, dropout. Underfitting: model too simple for the pattern. Both train and test accuracy are low. Fix: more features, more complex model, less regularization. Use learning curves (train and val score vs. training set size) to diagnose.

Q14. What is k-fold cross-validation and why use it?

INTERMEDIATE

ANSWER

Split data into k folds. Train on k-1, validate on the held-out fold. Repeat k times so each fold serves as validation once. Average the scores. Reduces variance in evaluation, uses all data for both train and validate. Stratified k-fold preserves class proportions — essential for imbalanced data. `TimeSeriesSplit` for ordered data (no future-leak).

Q15. How do you handle imbalanced classification?

INTERMEDIATE

ANSWER

(1) Don't use accuracy — use precision/recall/F1, ROC-AUC, PR-AUC. (2) Resample: SMOTE (oversample minority), RandomUnderSampler. (3) `class_weight='balanced'` in sklearn — penalizes majority class errors more. (4) Threshold tuning: optimize decision threshold for F-beta or business cost. (5) Use models robust to imbalance: tree ensembles often outperform linear models on imbalanced data.

Q16. Design a feature engineering pipeline for tabular data.

ADVANCED

ANSWER

Use `ColumnTransformer` to apply different transforms per column type. Numeric: impute (median) → scale (`StandardScaler/RobustScaler`). Categorical: impute (`mode/missing`) → `OneHot` or `TargetEncoder`. Datetime: extract `year/month/day_of_week/is_weekend`. Text: TF-IDF or embeddings. Wrap in a Pipeline so train/predict use identical transforms. Persist the full Pipeline, not individual steps.

CODE

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

num_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('sc', StandardScaler()),
])
cat_pipe = Pipeline([
    ('imp', SimpleImputer(strategy='most_frequent')),
    ('oh', OneHotEncoder(handle_unknown='ignore')),
])
prep = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols),
])
full = Pipeline([('prep', prep), ('model', model)])
```

Q17. Compare bagging, boosting, and stacking.

ADVANCED

ANSWER

Bagging (Random Forest): train many models in parallel on bootstrap samples; average predictions. Reduces variance. Boosting (XGBoost, LightGBM): sequentially train models, each focusing on previous errors. Reduces bias. Usually wins on tabular data. Stacking: train multiple diverse base models; train a meta-model on their predictions. Most flexible, most complex. Modern interview answer: 'I'd start with LightGBM/XGBoost for tabular — they win Kaggle competitions and require less feature engineering.'

Q18. How would you design an ML inference service for low latency?

ADVANCED

ANSWER

(1) Pre-load model at startup, not per request (use FastAPI startup events). (2) Batch requests with a micro-batching window (e.g. 10ms). (3) Use ONNX or TorchScript for faster inference than raw PyTorch/sklearn. (4) GPU only if helpful — for small models CPU + multi-process scales better. (5) Cache: Redis or in-process LRU for repeated inputs. (6) Async I/O for upstream/downstream. (7) Monitor: p50/p95/p99 latency, error rate, drift. (8) Feature store for consistency between training and serving features.

Q19. What is training-serving skew?

ADVANCED

ANSWER

When the features your model sees in production differ from what it was trained on — same code, different data behavior. Causes: (1) Different aggregation timezones/windows. (2) Live features include null defaults missing in training. (3) Encoding differences. (4) Training on historical data with leakage that's gone in real-time. Prevention: feature store, identical feature pipelines for train/serve, monitor feature distributions in production vs training.

Q20. Design a data pipeline that processes 1B events/day.

ADVANCED

ANSWER

Ingest: Kafka (~12k events/sec). Storage: lake on S3, partitioned by date/hour in Parquet (columnar, compressed). Processing: Spark or Flink for batch; Kafka Streams or Flink for stream. Orchestrate with Airflow. Schema in a registry (Confluent Schema Registry, AWS Glue). Data quality checks (Great Expectations / Soda) before downstream consumers. Catalog with Glue/Hive. Query layer: Athena/Trino for ad-hoc, Snowflake/Redshift for BI. Monitor freshness, completeness, schema drift.

Q21. Pickle vs joblib vs ONNX — when use each for model persistence?

ADVANCED

ANSWER

pickle: stdlib, easiest, Python-only, version-sensitive. Risk: can execute arbitrary code on load. joblib: optimized for NumPy arrays, faster on big sklearn models. Same security risks. ONNX: framework-neutral (export from PyTorch/sklearn/tf, load in C++/Java/JS), faster inference, production-grade. Use ONNX for serving across services or to a non-Python runtime. Use joblib for quick Python-to-Python persistence during development.

Q22. How would you A/B test a new ML model in production?

ADVANCED

ANSWER

(1) Shadow mode first: run new model alongside old, log predictions, don't act on them. Verify offline metrics match. (2) Canary: 1-5% of traffic to new model. Monitor business metrics (not just ML metrics — clicks, conversion, revenue). (3) Statistical significance: compute sample size needed for your effect size; don't stop early based on noise. (4) Gradual ramp: 5% → 25% → 50% → 100%. (5) Have a kill switch — feature flag to revert instantly. (6) Look for segment-specific regressions, not just overall.

Q23. Explain feature stores.

ADVANCED

ANSWER

A feature store is a centralized system for managing features across training and serving. Key capabilities: (1) Single source of truth — same feature definition in train and serve. (2) Backfill: compute historical features for new ML projects. (3) Point-in-time correctness: avoid future-leak in training. (4) Online store (Redis/DynamoDB) for low-latency serving; offline store (Parquet/Snowflake) for training. Tools: Feast (open-source), Tecton, AWS Feature Store, Databricks FS. Worth it when you have 5+ models sharing features.

Q24. What's a typical bug in time-series cross-validation?

ADVANCED

ANSWER

Using k-fold CV with shuffled splits — leaks future into past. Always use TimeSeriesSplit which keeps validation strictly after training. Also: feature engineering that touches future data (e.g. rolling means computed across all data, then split). Compute features only with past data available at prediction time. Walk-forward validation mirrors real-world deployment.

Q25. How do you monitor a model in production?

ADVANCED

ANSWER

(1) Performance monitoring: ground-truth comparison when available (delayed labels). Track precision/recall/AUC over time. (2) Drift monitoring: input feature distributions (PSI, KS test) and prediction distribution. (3) Data quality: nulls, ranges, schema. (4) Latency: p50/p95/p99. (5) Throughput, error rate, model version. (6) Business metrics that the model influences. (7) Alerts on significant drops. Tools: Evidently, Arize, WhyLabs, custom Prometheus + Grafana.

3. Django & FastAPI Web Frameworks

Both frameworks in depth — ORM/database patterns, async endpoints, dependency injection, auth, deployment, and the design tradeoffs interviewers love asking about.

Q1. Walk through what happens when a Django request hits the server.

BEGINNER

ANSWER

1) WSGI/ASGI server (gunicorn/uvicorn) hands HTTP request to Django. 2) Middleware chain runs request-side (auth, sessions, CSRF, etc.). 3) URL resolver matches path to a view. 4) View executes — may query ORM, render template, return Response. 5) Middleware chain runs response-side. 6) Response sent back. Each middleware is a class with `__call__` that wraps `get_response` — order matters.

DIAGRAM

```
Request
↓
[WSGI/ASGI]
↓
[Middleware: Security → Session → Auth → CSRF → ...]
↓
[URL Resolver] → urls.py → view
↓
[View] → ORM → Template/JSON
↓
[Middleware reverse order]
↓
Response
```

Q2. Django ORM: explain QuerySet laziness.

BEGINNER

ANSWER

QuerySets don't execute until evaluated. Chaining filters builds the query without hitting the database. Evaluation triggers: iteration, slicing (non-step), `len()`, `list()`, `bool()`, `repr()`. This means `User.objects.filter(active=True).filter(role='admin')` is ONE query, not two. But `for u in qs: ...` after `if qs:` runs it twice — cache with `list(qs)` or use `exists()`.

CODE

```
# Lazy — no DB query yet
qs = User.objects.filter(active=True)
qs = qs.filter(role='admin') # still no query
qs = qs.exclude(name='test') # still none

# Evaluation triggers query
list(qs) # query!
for u in qs: ... # query!
qs.count() # query
qs.exists() # cheap query (SELECT 1)
```

Q3. What's the N+1 query problem and how does Django solve it?

BEGINNER

ANSWER

Each related object triggers a separate query. Loading 100 blog posts then accessing `post.author` for each = 101 queries. Fix: `select_related` (SQL JOIN for ForeignKey/OneToOne — single query) or `prefetch_related` (separate query + Python join for ManyToMany/reverse FK).

CODE

```
# BAD — 1 + N queries
for post in Post.objects.all():
    print(post.author.name)    # query per post

# GOOD — 1 query
for post in Post.objects.select_related('author'):
    print(post.author.name)

# Many-to-many or reverse FK
for author in Author.objects.prefetch_related('books'):
    for book in author.books.all():
        print(book.title)
```

Q9. Why is FastAPI so fast?

BEGINNER

ANSWER

Three reasons: (1) Starlette under the hood — pure ASGI, async-first, no synchronous baggage. (2) Pydantic for validation — compiled (v2 uses Rust). (3) uvicorn server with uvloop. No middleware-per-request overhead like Django. Note: 'fast' is mostly the framework overhead — your DB and downstream calls still dominate real latency.

Q10. How do you define a FastAPI endpoint and what does the type hint do?

BEGINNER

ANSWER

Decorator + async function + Pydantic types. The type hints are not just documentation — FastAPI uses them to (1) validate input and generate 400 on bad data, (2) auto-generate OpenAPI schema, (3) provide IDE autocomplete. Path params, query params, body — all distinguished by type and signature position.

CODE

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post('/items/{item_id}')
async def create_item(item_id: int, item: Item, q: str = None):
    return {'id': item_id, 'item': item, 'q': q}

# item_id from URL path (int validation)
# item from body (Item validation)
# q from query string (optional)
```

Q4. select_related vs prefetch_related — when each?

INTERMEDIATE

ANSWER

`select_related`: SQL JOIN. Use for ForeignKey and OneToOne (forward direction). One query, but wide rows.
`prefetch_related`: two queries — main, then 'IN (...)' for related — joined in Python. Use for ManyToMany and reverse ForeignKey. You can stack: `.select_related('author').prefetch_related('tags')`. Both can chain through relations: `select_related('author__profile')`.

Q5. How does Django middleware work?

INTERMEDIATE

ANSWER

Middleware is a class (or factory) that wraps the request/response cycle. Defined in `settings.MIDDLEWARE` — order matters. Each middleware receives `get_response` and returns a callable. Inside, code BEFORE `self.get_response(request)` runs on the way in; code AFTER runs on the way out. `process_view`, `process_exception`, `process_template_response` give hooks for specific phases.

CODE

```
class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        duration = time.time() - start
        response['X-Duration'] = str(duration)
        return response
```

Q6. Django REST Framework: ViewSet vs APIView vs generic views.

INTERMEDIATE

ANSWER

`APIView`: most explicit — you write `get/post/put/delete` methods. Full control. Generic views (`ListAPIView`, `RetrieveAPIView`, etc.): pre-built CRUD with mixins (`ListModelMixin`, etc.). `ViewSet`: groups multiple actions (`list`, `retrieve`, `create`, `update`, `destroy`) into one class, wired to a router. Best for CRUD APIs — minimal boilerplate. `ModelViewSet` = full CRUD on one model.

CODE

```
from rest_framework import viewsets
from rest_framework.routers import DefaultRouter

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    permission_classes = [IsAuthenticated]

router = DefaultRouter()
router.register('books', BookViewSet)
urlpatterns = router.urls
# Auto-generates: /books/, /books/<id>/, GET/POST/PUT/DELETE
```

Q7. DRF Serializer vs ModelSerializer.

INTERMEDIATE

ANSWER

`Serializer`: define every field explicitly. Use for custom outputs not tied to a model, complex validation, nested write logic. `ModelSerializer`: auto-generates fields from a Model — sets fields, types, validators. Use for typical CRUD. Override `.create()/update()` for nested write logic. Both: `validate_<field>()` for field-level validation, `validate()` for cross-field.

Q8. Django migrations — how do they actually work?

INTERMEDIATE

ANSWER

Migrations are Python files describing schema changes. `makemigrations` introspects model differences and generates migration files. `migrate` applies them in order, recording history in `django_migrations` table. Each migration: dependencies (which others must run first), operations (`CreateModel`, `AddField`, `RunPython`, etc.). For raw SQL, use `RunSQL` with `reverse_sql`. NEVER delete an applied migration file in production — `sqlmigrate` to inspect what runs.

Q11. FastAPI: sync vs async path functions — what's the difference?

INTERMEDIATE

ANSWER

`async def`: runs on the event loop. Use for await-able DB calls, HTTP calls. `def (sync)`: FastAPI runs it in a thread from a pool, so it doesn't block the loop. Verdict: use `async def` for I/O-bound work with async libraries. Use plain `def` for blocking libraries (e.g. SQLAlchemy sync, requests). MIXING wrong: `async def` calling a sync blocking function freezes the entire loop.

Q12. Explain Pydantic v2 and what changed from v1.

INTERMEDIATE

ANSWER

v2 (released 2023) reimplements core in Rust → 5-50x faster validation. Key API changes: `BaseModel.model_dump()` replaces `.dict()`; `model_validate()` replaces `parse_obj()`; Field syntax tweaks; `@validator` → `@field_validator`; `@root_validator` → `@model_validator`; `from_orm()` → `model_validate()` with `from_attributes=True`. Strict mode (no type coercion) opt-in. Migration script: `bump-pydantic`.

CODE

```
from pydantic import BaseModel, Field, field_validator

class User(BaseModel):
    name: str = Field(min_length=1)
    age: int = Field(ge=0, le=150)
    email: str

    @field_validator('email')
    @classmethod
    def email_lower(cls, v):
        return v.lower()

u = User.model_validate({'name': 'a', 'age': 30, 'email': 'A@b.com'})
data = u.model_dump()
```

Q13. FastAPI dependency injection — how and why?

INTERMEDIATE

ANSWER

`Depends()` resolves a callable, runs it, passes the result as a parameter. Used for: DB sessions, auth, pagination, shared logic. Dependencies can have their own dependencies — cached per request by default. yield-based dependencies handle setup/teardown (replaces 'finally' patterns).

CODE

```
from fastapi import Depends, HTTPException

async def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

async def current_user(token: str = Header(...),
                        db = Depends(get_db)):
    user = db.query(User).filter_by(token=token).first()
    if not user:
        raise HTTPException(401)
    return user

@app.get('/me')
async def me(user = Depends(current_user)):
    return user
```

Q14. How do you handle background tasks in FastAPI?

INTERMEDIATE

ANSWER

Three levels: (1) `BackgroundTasks`: simple in-request work (send email after response). Runs in the same process after response is sent. (2) Celery: for heavy work, retries, scheduling. Tasks run in separate workers. (3) `APScheduler` or `arq`: lighter than Celery, async-friendly. For long-running streams, see `WebSockets` / `Server-Sent Events`.

CODE

```
from fastapi import BackgroundTasks

def send_email(to: str, subject: str):
    # blocking is fine – runs after response sent
    smtp.send(to, subject)

@app.post('/signup')
async def signup(user: UserIn, bg: BackgroundTasks):
    save_user(user)
    bg.add_task(send_email, user.email, 'Welcome!')
    return {'ok': True}
```


Q15. FastAPI WebSockets — basic pattern.

INTERMEDIATE

ANSWER

Decorator `@app.websocket`. Accept the connection, then receive/send in a loop until disconnect. State management: hold a `ConnectionManager` that tracks active sockets and broadcasts. For scale: WebSockets are stateful — use Redis pub/sub or a message broker to fan out across server instances.

CODE

```
from fastapi import WebSocket, WebSocketDisconnect

active = set()

@app.websocket('/ws')
async def ws(websocket: WebSocket):
    await websocket.accept()
    active.add(websocket)
    try:
        while True:
            msg = await websocket.receive_text()
            for ws in active:
                await ws.send_text(msg)
    except WebSocketDisconnect:
        active.remove(websocket)
```

Q16. Auth strategies: session, JWT, OAuth — when use each?

INTERMEDIATE

ANSWER

Sessions (server-stored, cookie ID): traditional web app, server-rendered. Easy to revoke. JWT (signed token): stateless APIs, microservices, mobile. Hard to revoke before expiry — keep short-lived (15min) + refresh tokens. OAuth 2.0 / OIDC: 'Sign in with Google'. You delegate auth to a provider; receive tokens. Use libs (authlib, python-social-auth, fastapi-users). NEVER roll your own crypto — use proven libs.

Q17. Compare Django ORM, SQLAlchemy, and Tortoise ORM.

INTERMEDIATE

ANSWER

Django ORM: tightly coupled to Django, synchronous by default (async since 4.1 but limited). Pythonic, simple migrations. SQLAlchemy: framework-agnostic, most powerful — Core (SQL builder) + ORM. async support since 1.4. Best for complex queries. Tortoise: async-native, Django-like API. Great fit for FastAPI. Async SQLAlchemy is usually the right answer for new async services; Django ORM if you're already on Django.

Q18. Django: how would you scale a slow database query?

ADVANCED

ANSWER

(1) Profile first: `django-debug-toolbar`, `django-silk`, or `django.db.connection.queries`. (2) Add indexes — covering indexes for filter+order. (3) `select_related/prefetch_related` for N+1. (4) `only()/defer()` to fetch fewer columns. (5) Database denormalization or computed columns. (6) Cache: per-view cache, low-level cache framework with Redis. (7) Read replicas via `DATABASE_ROUTERS`. (8) Move heavy reports to async jobs + materialized views. (9) Last resort: raw SQL with `.raw()` or `.extra()`.

Q19. How do Django channels work, and when use them over FastAPI?

ADVANCED

ANSWER

Django Channels adds ASGI support — WebSockets, long-polling, async views. Uses a 'channel layer' (typically Redis) to dispatch messages between consumers. Best when: you already have a Django app and want real-time features (chat, notifications) without rewriting. FastAPI: better when starting fresh and async-first, or you need pure-API service performance. Both can scale, but Django's ORM is sync-first, so async views still hit a sync ORM under the hood — Channels doesn't fix that.

Q20. How do you test FastAPI endpoints?

ADVANCED

ANSWER

Use `TestClient` from `fastapi.testclient` (wraps `httpx`). Test fixtures: a test DB (sqlite or per-test schema) wired via `Depends` override. `app.dependency_overrides[get_db] = lambda: test_db`. `pytest-asyncio` for async tests with `httpx.AsyncClient`. Avoid testing through the real DB on every test — use rollback transactions or transactional fixtures.

CODE

```
from fastapi.testclient import TestClient

def get_test_db():
    yield TestSession()

app.dependency_overrides[get_db] = get_test_db
client = TestClient(app)

def test_create():
    r = client.post('/items', json={'name': 'x'})
    assert r.status_code == 200
    assert r.json()['name'] == 'x'
```

Q21. Deployment: gunicorn vs uvicorn — which workers? How many?

ADVANCED

ANSWER

Django (sync): gunicorn with sync workers — N workers, ~2*CPU cores. FastAPI (async): gunicorn -k uvicorn.workers.UvicornWorker (each is async event-loop). Workers = CPU count usually enough — one worker handles thousands of connections via async. For mixed I/O+CPU, more workers. Always front with nginx for TLS, static files, request limits. uvicorn alone is fine for dev; gunicorn-managed gives graceful restarts, worker recycling.

Q22. How do you implement file uploads safely?

ADVANCED

ANSWER

(1) Stream to disk/S3, don't read full file into memory (await file.read() = OOM risk on large uploads). FastAPI: UploadFile already streams. (2) Validate content-type AND magic bytes (don't trust extension). (3) Cap file size — webserver level (nginx client_max_body_size) and app level. (4) Store outside webroot OR in S3 with restricted ACLs. (5) Generate new filenames (UUID) — don't trust user-supplied names (path traversal). (6) Scan for malware if user-facing. (7) Serve via CDN with content-disposition: attachment to prevent inline JS execution.

CODE

```
from fastapi import UploadFile, File, HTTPException

@app.post('/upload')
async def upload(f: UploadFile = File(...)):
    if f.content_type not in {'image/jpeg', 'image/png'}:
        raise HTTPException(415)
    name = f'{uuid4()}_{Path(f.filename).suffix}'
    async with aiofiles.open(f'/uploads/{name}', 'wb') as out:
        while chunk := await f.read(64 * 1024):
            await out.write(chunk)
    return {'name': name}
```

Q23. Async SQLAlchemy with FastAPI — what's the gotcha?

ADVANCED

ANSWER

Sessions must NOT cross await boundaries between requests. Each request gets its own AsyncSession via dependency. Don't store sessions globally. Watch out for: (1) Lazy loading — async-only, must use selectinload/joinedload. (2) Engine creation at module import: use create_async_engine once. (3) DB driver must be async (asyncpg for Postgres, aiomysql for MySQL). sync drivers won't work.

CODE

```
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine

engine = create_async_engine('postgresql+asyncpg://...')
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db():
    async with SessionLocal() as session:
        yield session

@app.get('/users/{id}')
async def get_user(id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == id))
    return result.scalar_one_or_none()
```

Q24. How do you secure a public API?

ADVANCED

ANSWER

(1) HTTPS only (HSTS header). (2) Auth: API keys (basic), OAuth 2 (delegated), JWT with short TTL + refresh. (3) Rate limiting per key/IP (slowapi, fastapi-limiter with Redis). (4) Input validation — Pydantic handles 95%, but validate sizes (length, count) explicitly. (5) CORS: whitelist origins, never '*' in prod. (6) Don't leak in errors — generic 500 to clients, full trace to logs. (7) Audit logging for sensitive ops. (8) Security headers (CSP, X-Frame-Options). (9) Pen-test or use OWASP ZAP. (10) Dependencies: pip-audit, dependabot for CVEs.

Q25. Django: implementing a feature flag system.

ADVANCED

ANSWER

Options: (1) django-waffle — mature, supports flags/switches/samples per-user. (2) Self-built: model FeatureFlag(name, enabled, percentage, users); middleware/utility checks. (3) External: LaunchDarkly, Unleash. Consider: per-user targeting, gradual rollout %, A/B test integration. Cache flag lookups (every request reading DB is too slow). Default 'off' — flags should be safe to forget.

Q26. Django vs FastAPI — when do you genuinely pick which?

ADVANCED

ANSWER

Django for: CMS-like apps, admin panel out of the box, batteries-included (auth, forms, templates, ORM, migrations), full-stack web app. FastAPI for: API-only services, async-heavy I/O, microservices, ML serving, type-safety obsession. Hybrid: FastAPI for the public API, Django for internal admin. Don't pick FastAPI just for 'speed' — most apps aren't framework-bound; pick for the developer experience that matches your team and product.

Q27. Database migrations in production — what's the playbook?

ADVANCED

ANSWER

(1) Backward-compatible migrations only: add nullable column → deploy code that reads/writes both → backfill → make non-nullable. NEVER drop a column in the same release that stops using it. (2) Large tables: split into small migrations, use pt-online-schema-change or gh-ost for MySQL, CONCURRENTLY for Postgres index/constraint. (3) Always test against prod-like data volume. (4) Reversible migrations (down() defined) for rollback. (5) Lock-aware: long ALTER blocks writes. (6) Run migrations BEFORE deploying code that needs them (zero-downtime). (7) Have a runbook + DBA review for risky ones.

4. Cheatsheets

Quick-reference tables. Print and pin.

Async I/O Quick Reference

Pattern / Need	Use This
Run async program from sync entry point	<code>asyncio.run(main())</code>
Run many coroutines, wait for all	<code>await asyncio.gather(*coros)</code>
Wait for first to finish	<code>await asyncio.wait(..., return_when=FIRST_COMPLETED)</code>
Stream results as they complete	<code>asyncio.as_completed(coros)</code>
Structured concurrency (3.11+)	<code>async with asyncio.TaskGroup() as tg: tg.create_task(...)</code>
Run blocking function	<code>await asyncio.to_thread(blocking_fn, args)</code>
Run CPU-bound function	<code>await loop.run_in_executor(ProcessPoolExecutor(), fn)</code>
Yield to loop briefly	<code>await asyncio.sleep(0)</code>
Cancel a task	<code>task.cancel(); await task with except CancellationError</code>
Protect from cancellation	<code>await asyncio.shield(coro)</code>
Timeout	<code>async with asyncio.timeout(5): ... (3.11+)</code>
Per-task context	<code>contextvars.ContextVar('name')</code>

Pattern / Need	Use This
Production event loop	<code>uvloop.install()</code>
Limit concurrent in-flight	<code>asyncio.Semaphore(n)</code>

Pandas Quick Reference

What You Want	Code
Read CSV with types	<code>pd.read_csv(path, dtype={'a': 'int32'}, parse_dates=['d'])</code>
Read Parquet (fastest)	<code>pd.read_parquet(path, columns=['a', 'b'])</code>
Select columns	<code>df[['a', 'b']]</code> or <code>df.loc[:, ['a', 'b']]</code>
Filter rows	<code>df[df['x'] > 0]</code> or <code>df.query('x > 0 and y < 10')</code>
Group + aggregate	<code>df.groupby('city').agg({'sales': ['mean', 'sum']})</code>
Pivot table	<code>df.pivot_table(index='date', columns='item', values='qty', aggfunc='sum')</code>
Merge / join	<code>pd.merge(df1, df2, on='key', how='left')</code>
Concat (stack)	<code>pd.concat([df1, df2], axis=0, ignore_index=True)</code>
Apply function (vectorized first!)	<code>df['x'].map(fn)</code> or <code>df.apply(fn, axis=1)</code>
Drop duplicates	<code>df.drop_duplicates(subset=['a', 'b'])</code>
Handle missing	<code>df.fillna(0)</code> or <code>df.dropna(subset=['x'])</code>
Rolling stats	<code>df['x'].rolling(window=7).mean()</code>
Datetime ops	<code>df['dt'].dt.year / .dt.dayofweek / .dt.floor('H')</code>
Memory usage	<code>df.memory_usage(deep=True)</code>
Reduce memory	<code>df['cat'] = df['cat'].astype('category')</code>
Write Parquet	<code>df.to_parquet(path, compression='snappy')</code>

scikit-learn Quick Reference

Need	Tool
Train/test split	<code>train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)</code>
Scale features	<code>StandardScaler()</code> / <code>MinMaxScaler()</code> / <code>RobustScaler()</code>
Encode categories	<code>OneHotEncoder(handle_unknown='ignore')</code>
Impute missing	<code>SimpleImputer(strategy='median')</code>
Pipeline (avoid leakage)	<code>Pipeline([('prep', ...), ('model', ...)])</code>
Different transforms per col	<code>ColumnTransformer([('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols)])</code>
k-fold CV	<code>cross_val_score(pipe, X, y, cv=5, scoring='f1')</code>
Stratified for imbalanced	<code>StratifiedKFold(n_splits=5)</code>

Need	Tool
Time series CV	<code>TimeSeriesSplit(n_splits=5)</code>
Hyperparam search	<code>GridSearchCV(pipe, params, cv=5)</code> / <code>RandomizedSearchCV</code>
Best tabular model start	<code>from lightgbm import LGBMClassifier</code>
Confusion / report	<code>classification_report(y_true, y_pred)</code>
ROC-AUC for ranking	<code>roc_auc_score(y_true, y_proba)</code>
Imbalanced	<code>class_weight='balanced' + SMOTE from imblearn</code>
Save model	<code>joblib.dump(pipe, 'model.pkl')</code>
Production-grade serving	Export to ONNX → load in onnxruntime

Django/FastAPI Decision Matrix

Scenario	Better Pick
Building CMS / admin-heavy app	Django (free admin, auth, ORM)
Pure JSON API, low latency	FastAPI
ML model serving	FastAPI (async + Pydantic for I/O schemas)
Real-time / WebSockets / chat	FastAPI or Django Channels
Full-stack with server-rendered templates	Django
Microservice or many small services	FastAPI
Heavy ORM with relational data	Django ORM or SQLAlchemy
Existing Django app, need WebSockets	Add Channels
Team unfamiliar with async	Django (sync-first is simpler)
Auto-generated OpenAPI / SDK	FastAPI (built-in)
Lots of forms, validation, CSRF	Django
GraphQL service	Strawberry on FastAPI, or Graphene on Django
Massive scale, multi-tenant SaaS	Either — depends on team skill, not framework
Rolling out gradually from monolith	FastAPI for new endpoints, Django stays